Assignment 3

Pragya Singhal ,CU24220050

1. **Explanation:** A plain JavaScript variable does not trigger a UI re-render in React. When you change a normal variable's value, React's virtual DOM is not notified, and the component function does not re-run, meaning the screen remains outdated. In contrast, useState stores state and tells React to re-render the component whenever the value updates.

```
import React, { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Counter: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
}
```

2. **Explanation:** Calling setCount directly inside the render body forces a re-render on every execution, which immediately calls setCount again, creating an infinite loop. Moving it into an event handler or useEffect solves this.

```
import React, { useState } from 'react';

export default function CounterFixed() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment Safely</button>
    </div> ); }
```

Q1: Counter Component

Count: 1

Increment Decrement

3. **Explanation:** The empty dependency array [] ensures that the useEffect hook runs strictly once when the component mounts, preventing an infinite fetch loop.

```
import React, { useState, useEffect } from 'react';

export default function FetchData() {
  const [posts, setPosts] = useState([]);


  useEffect(() => {
    // Fetching data on component mount
    fetch('/api/posts')
      .then(res => res.json())
      .then(data => setPosts(data))
      .catch(err => console.log(err));
  }, []); // Empty dependency array ensures it runs only once
  return (
    <ul>
      {posts.map(post => <li key={post.id}>{post.title}</li>)}
    </ul>  ); }
```

Q3: useEffect Data Fetching

- First Post
- Second Post

4. Why Empty Dependency Array [] is Necessary

The empty dependency array [] tells React to run the effect only once when the component initially mounts, preventing infinite API requests on re-render.

```
import React, { useState } from 'react';

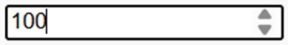
function PriceInput({ price, setPrice }) {
  return <input type="number" value={price} onChange={e => setPrice(e.target.value)} />;  }

function PriceDisplay({ price }) {
  return <p>Price: {price}</p>; }

export default function PriceApp() {
  const [price, setPrice] = useState(0);
  return (
    <div>
      <PriceInput price={price} setPrice={setPrice} />
```

```
    <PriceDisplay price={price} />
  </div>  ); }
```

Q4: Lifting State Up



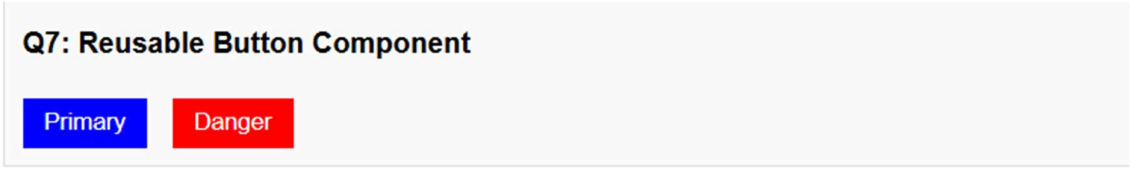
Current Price: 100

```
5. import { useState, useEffect } from 'react';
export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved !== null ? JSON.parse(saved) : initialValue;
  });
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);
  return [value, setValue]; }
6. {items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

Q6: List Rendering with Keys

- Apple
- Banana

```
7. import React from 'react';
import './Button.css'; // Optional CSS file
export default function Button({ variant, children, onClick }) {
  const className = variant === 'danger' ? 'btn-danger' : 'btn-primary';
  return (
    <button className={className} onClick={onClick}>
      {children}
    </button>  ); }
```



Question 8: Preventing Child Re-renders with React.memo

Explanation By default in React, updating a parent's state forces all of its child components to re-render, even if the child's own props remain unchanged.

```
import React, { useState } from 'react';

const ChildCard = React.memo(function ChildCard({ title }) {

 console.log('ChildCard rendered');

 return <h3>{title}</h3>; });

export default function ParentDashboard() {

 const [tick, setTick] = useState(0);

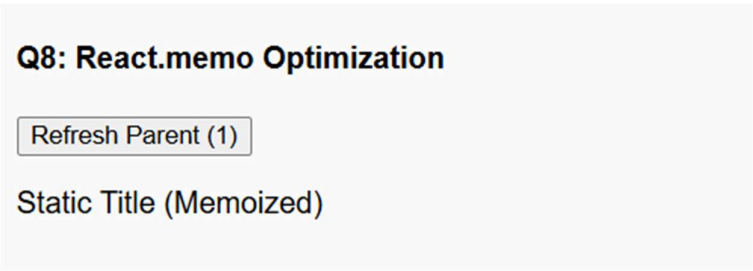
 return (

  <div>

   <button onClick={() => setTick(tick + 1)}>Refresh ({tick})</button>

   <ChildCard title="Static Title" />

  </div> ); }
```




```
9. import React from 'react';

import { BrowserRouter, Routes, Route, Link, useParams } from 'react-router-dom';

function Home() { return <h2>Home Page</h2>; }

function Products() { return <h2>Products Page</h2>; }

function ProductDetail() {

 const { id } = useParams();

 return <h2>Product ID: {id}</h2>;

}


export default function AppRouter() {
```

```
 return (
  <BrowserRouter>
   <nav>
    <Link to="/">Home</Link> | <Link to="/products">Products</Link> | <Link to="/products/1">Product 1</Link>
   </nav>
   <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/products" element={<Products />} />
    <Route path="/products/:id" element={<ProductDetail />} />
   </Routes>
  </BrowserRouter> ); }
```

10.
```
import React, { useState } from 'react';
export default function UserForm() {
 const [formData, setFormData] = useState({
  name: '',
  email: '',
  phone: ''
 });
 const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({ ...prev, [name]: value }));
 };
 return (
  <form>
   <input name="name" value={formData.name} onChange={handleChange} placeholder="Name" />
   <input name="email" value={formData.email} onChange={handleChange} placeholder="Email" />
   <input name="phone" value={formData.phone} onChange={handleChange} placeholder="Phone" />
  </form> ); }
```

